



SEMICONDUCTOR
ACADEMY
LEARN. DESIGN. PARTICIPATE.

How Computers Execute Code

From human-readable programming languages to billions of physical operations per second inside the silicon, discover the journey of a software instruction.

THE TRANSLATION JOURNEY

Before a computer can run your code, human-readable instructions must be translated into the only language hardware understands: binary machine code.

01

High-Level Code

Languages like C, Python, or Java are designed for human readability but cannot be directly executed by hardware.

02

Compilation / Interpretation

Compilers translate high-level code into assembly and then binary machine code. Interpreters execute on the fly during runtime.

03

Executable on Disk

Executables store binary instructions permanently on disk until the OS loads them into RAM to begin execution.



LOADING INTO MEMORY (RAM)

When a program is launched, the OS transitions the binary executable from storage into active system memory.



Program Instructions

Compiled binary code loaded into a read-only memory segment.

Stack

Temporary data: function parameters, return addresses, local variables.

Heap

Dynamic allocation pool — variables created and destroyed at runtime.

Memory Layout

Code Segment

Stack ↑

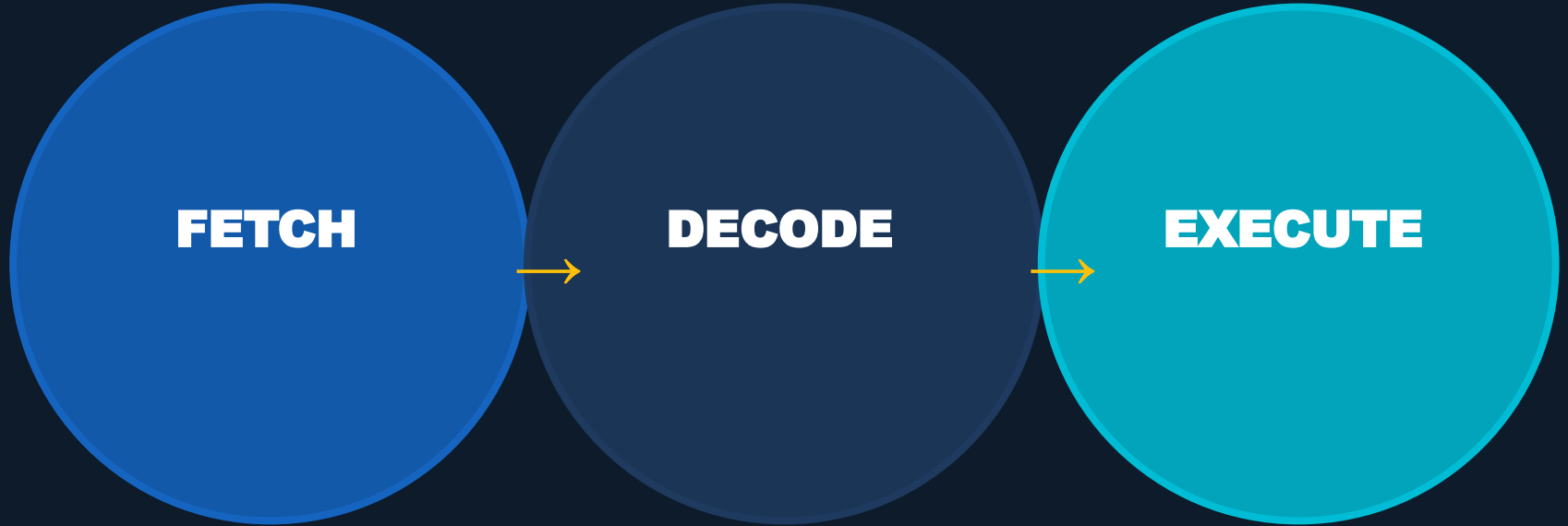
Free Memory

Heap ↓



THE HEART OF EXECUTION

The CPU continuously runs the Fetch-Decode-Execute cycle billions of times per second.



Retrieve next instruction from RAM using the Program Counter

Control Unit interprets binary patterns into electrical signals

ALU and functional units perform the actual computation

↻ Repeats billions of times per second



STEP 1 - FETCHING THE INSTRUCTION

Retrieving from RAM

The CPU locates and retrieves the next instruction to be executed from the system memory.



Program Counter (PC)

A specialized register that holds the memory address of the very next instruction to be executed.



Instruction Register (IR)

Once fetched from RAM, the binary instruction is temporarily stored here for immediate processing.



STEP 2 - DECODING THE INSTRUCTION

Translating Binary Signals

The raw binary instruction must be interpreted to determine what action the hardware needs to take.

Control Unit (CU)

The brain of the CPU that decodes binary patterns into electrical signals directing the hardware.

Operation Types

The CU determines whether the instruction requires arithmetic, logical comparisons, memory access, or data movement.

10110100 00101101 11000010 01101001 10001111 00110101 11100001 01011010



STEP 3 - EXECUTING THE INSTRUCTION

Decoded signals are sent to the functional units of the processor to perform the actual computation.



Arithmetic Logic Unit (ALU)

Performs mathematical calculations (addition, subtraction) and logical comparisons (AND, OR, NOT).



Registers

Tiny, ultra-fast storage cells inside the CPU that hold immediate data and results for the ALU.



Memory Writeback

The final result of execution is written back to a register or sent to main RAM for storage.



OPTIMIZING CPU PERFORMANCE

Modern processors use advanced hardware techniques to bypass physical bottlenecks and accelerate execution.



Cache Memory

Stores frequently used data extremely close to the CPU, drastically reducing time spent waiting for slower RAM.



Pipelining

Processes multiple instructions simultaneously at different stages of the fetch-decode-execute cycle.



Out-of-Order Execution

Executes independent instructions as soon as resources are free, rather than waiting for preceding delayed instructions.



ADVANCED PROCESSOR ARCHITECTURE

Parallelism & Prediction

To push performance limits, modern chips rely on intelligent prediction and multi-core designs.



Branch Prediction

The CPU guesses the future path of conditional statements (like IF/ELSE) to pre-fetch instructions and avoid idle delays.

This technique keeps the pipeline full and avoids costly stalls when code branches.



Multi-Core Processing

Running multiple independent instruction streams in parallel across separate physical cores.

Modern CPUs may have 8, 16, or even 64+ cores, each capable of running its own execution pipeline.



THE ROLE OF THE OPERATING SYSTEM



While the CPU executes raw instructions, the Operating System (OS) acts as the master coordinator.

Memory Allocation

Manages the distribution of RAM to processes and reclaims memory when no longer needed.

Process Scheduling

Decides which process runs on the CPU and for how long, creating the illusion of parallel execution.

I/O Management

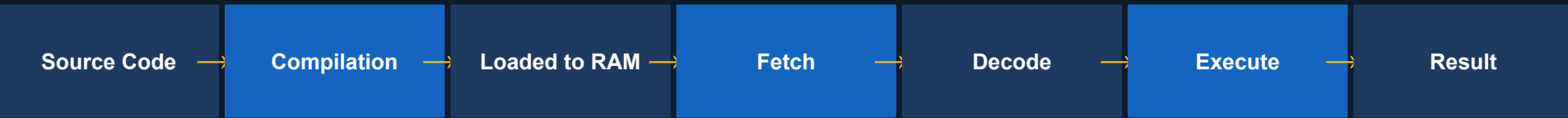
Handles all communication with input/output devices, abstracting hardware complexity from programs.

Conflict Prevention

Ensures multiple programs run smoothly without interfering with each other's memory or resources.



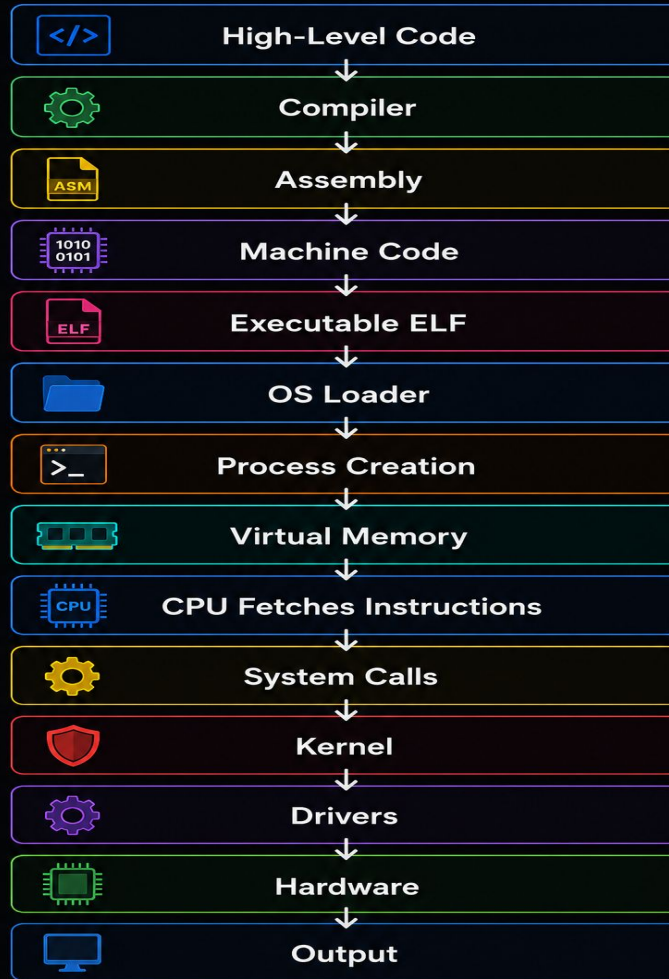
The Full Journey



Key Takeaways

- ▶ Every program you write is ultimately translated into binary and processed through the Fetch-Decode-Execute cycle.
- ▶ RAM holds code, stack, and heap segments — each serving a distinct role during runtime.
- ▶ The CPU's ALU, Control Unit, and registers work in concert to execute billions of operations per second.
- ▶ Modern CPUs use caching, pipelining, and out-of-order execution to maximize throughput.
- ▶ The Operating System orchestrates all these processes, ensuring stability across applications.





Q&A

Thank You

